

JOL ENERGY PVT. LTD. · SDE & DATA ANALYTICS
INTERN
INTERNSHIP TASK ROUND — DASHBOARD BUILD GUIDE

Claude Code Task Prompts

Complete build instructions for the Jol Energy BI Dashboard

*10 sequenced tasks · Each task is a complete, self-contained Claude Code prompt
Data files are pre-generated — focus entirely on building the UI and integrations*

11	5	~7h	✓
Tasks	Modules	Build time	Vercel deploy

How to use this document

Each task below contains a complete Claude Code prompt. Open Claude Code in your terminal (claude), paste the prompt exactly as written, and let it work. Run tasks in order — each one builds on the previous.

Task execution order

The tasks are numbered TASK-00 through TASK-10. Run them sequentially. Skipping a task will cause the next one to fail because the file structure and component dependencies are cumulative.

Task	What it builds	Time
TASK-00	Project scaffold: Vite, npm install, Tailwind, folder structure, data files	~15 min
TASK-09	Utility files: formatters, health score, CSV export (run before modules if needed)	~20 min
TASK-01	App shell: top bar, 5 tab navigation, health score ring	~30 min
TASK-02	Reusable components: KPICard, FunnelChart, LeadTable	~45 min
TASK-03	Module 1: Supplier pipeline	~60 min
TASK-04	Module 2: Buyer pipeline	~50 min
TASK-05	Module 3: Collection network geo-map	~60 min
TASK-06	Module 4: Market intelligence + margin calculator	~60 min
TASK-07	Module 5: AI insights engine (Claude API)	~60 min
TASK-08	Smart alerts panel + global polish	~40 min
TASK-10	Vercel deployment + README	~20 min

Before you start

1. Install Claude Code: `npm install -g @anthropic-ai/claude-code`
2. Make sure Node.js 18+ is installed: `node --version`
3. Have your Anthropic API key ready from console.anthropic.com
4. Confirm the 4 data files are in the same folder: `supplierLeads.js`, `buyerLeads.js`, `collectionPoints.js`, `marketPrices.js`
5. Open a terminal in the folder where you want to create the project
6. Run: `claude` (this opens the Claude Code CLI)
7. Paste the TASK-00 prompt and press Enter

Tips for Claude Code

- If Claude Code asks a clarifying question, answer briefly then remind it to proceed with the task.
- If an install step fails (npm ERR!), paste the error into Claude Code and say 'fix this and continue the task'.
- After each task, verify the dev server still starts with npm run dev before moving to the next task.
- If the map (TASK-05) shows grey boxes instead of pins, the Leaflet icon fix is the cause — remind Claude Code to apply the L.Marker.prototype.options.icon fix.
- For the AI module (TASK-07): the streaming fetch pattern must be exact. If you see a CORS error, check the anthropic-version header is '2023-06-01'.

TASK-00**Project scaffold & dependency setup****M0**
~15 min

Module	M0 · Foundation
Estimated time	~15 min
Prerequisites	Node.js 18+ installed · npm or yarn available · Vercel CLI (optional but recommended)

DELIVERS

- Vite + React 18 project
- All dependencies installed
- Tailwind configured
- Folder structure created
- Data files placed in src/data/

CONTEXT NOTE

Run this first. Every subsequent task assumes this structure exists. Claude Code should scaffold the project, install deps, and verify the dev server starts without errors.

PROMPT — copy everything below this line into Claude Code

Create a new Vite + React 18 project called jol-dashboard in the current directory.

```
Run: npm create vite@latest jol-dashboard -- --template react
```

Then install all dependencies:

```
cd jol-dashboard
```

```
npm install recharts leaflet react-leaflet papaparse @anthropic-ai/sdk
```

```
npm install -D tailwindcss postcss autoprefixer
```

```
npx tailwindcss init -p
```

Configure tailwind.config.js to scan src/**/*.{js,jsx}.

In src/index.css replace all content with:

```
@tailwind base;
```

```
@tailwind components;
```

```
@tailwind utilities;
```

Create the following empty directories:

```
src/components/
```

```
src/data/
```

```
src/utils/
```

Copy the four pre-generated data files into src/data/:

```
supplierLeads.js  
buyerLeads.js  
collectionPoints.js  
marketPrices.js
```

Add leaflet CSS import to src/main.jsx:

```
import 'leaflet/dist/leaflet.css'
```

Create a vercel.json in the project root:

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

Create a .env file in the project root with a placeholder:

```
VITE_ANTHROPIC_KEY=your_key_here
```

Add .env to .gitignore.

Verify the dev server starts with `npm run dev` and the default Vite page renders.

Report any errors.

TASK-01**Global layout, navigation & health score ring****M0**
~30 min

Module	M0 · Foundation
Estimated time	~30 min
Prerequisites	TASK-00 complete

DELIVERS

- App.jsx with 5-tab navigation
- Top bar with logo + alert bell
- Pipeline health score ring (HealthScore.jsx)
- Active tab border in module colour
- URL updates with ?tab= param on navigation

CONTEXT NOTE

The health score ring is the centrepiece of the top bar. Formula: $(contactRate \times 0.30) + (qualRate \times 0.25) + (velocityScore \times 0.20) + (coverageScore \times 0.25)$. Score 0–100, rendered as an SVG arc ring.

PROMPT — copy everything below this line into Claude Code

In src/App.jsx build the complete application shell with the following:

1. TOP BAR (full width, sticky)

Left: 'JOL ENERGY' logo text in dark navy bold + 'Dashboard' in blue

Center: 5 tab buttons — Suppliers | Buyers | Collection | Market | AI Insights

Right: Pipeline Health Score ring (component below) + alert bell icon (badge with count)

2. TAB NAVIGATION

Use React useState for activeTab (default 'suppliers').

Active tab: 2px bottom border in module colour:

suppliers = #185FA5 (blue)

buyers = #0A7864 (teal)

collection= #B45309 (amber)

market = #534AB7 (purple)

ai = #A32D2D (red)

On tab change, update URL with window.history.pushState({}, '', '?tab='+tab).

On page load, read ?tab= from URL and set initial active tab.

3. CREATE src/components/HealthScore.jsx

Props: score (0–100), size (default 64)

Render an SVG ring gauge:

- Background circle: stroke #E6F1FB, strokeWidth 6
- Foreground arc: stroke colour depends on score:
 - 0-40 = #E24B4A (red)
 - 41-70 = #EF9F27 (amber)
 - 71-100 = #1D9E75 (teal)
- Arc length = (score/100) * circumference (strokeDasharray trick)
- Centre text: score number, 14px bold
- Below text: 'Health' label, 10px grey

Import from src/data/supplierLeads.js { supplierStats } and compute:

```
contactRate = (Contacted+Qualified+Interested+LOI) / total
qualRate    = (Qualified+Interested+LOI) /
max(Contacted+Qualified+Interested+LOI, 1)
coverageScore = min(supplierStats.totalKg / 10000, 1) # 10T target
velocityScore = 0.6 # fixed baseline for dummy data
healthScore = Math.round(contactRate*30 + qualRate*25 + velocityScore*20 +
coverageScore*25)
```

4. MAIN CONTENT AREA

Below top bar, render the active module component (import stubs for now – just show a placeholder div for each).

5. GLOBAL STYLES

Font: system-ui / sans-serif. Background: #F8F9FA. Cards: white with 1px #E0E0E0 border, 8px radius.

All colours must use the Tailwind config or inline style – no hardcoded colour strings in JSX except hex values.

Ensure the app compiles and the tab navigation visually switches between placeholder panels.

TASK-02**Reusable components: KPICard, FunnelChart, LeadTable****M0**
~45 min

Module	M0 · Foundation
Estimated time	~45 min
Prerequisites	TASK-01 complete

DELIVERERS

- KPICard.jsx with animated count-up + provenance tooltip
- FunnelChart.jsx (custom SVG funnel)
- LeadTable.jsx (sortable, filterable, expandable rows)
- useCountUp hook

CONTEXT NOTE

These three components are used across multiple modules. Build them once correctly here. The count-up animation must use requestAnimationFrame. The lead table must support external filter state passed as props.

PROMPT — copy everything below this line into Claude Code

Create the following three reusable components in src/components/.

```
# — COMPONENT 1: KPICard.jsx —————
Props: value (string|number), label (string), subtext (string),
       color (hex, default '#185FA5'), source (string), animateOnMount (bool)

Render: white card, 12px radius, 1px #E0E0E0 border, 16px padding.
Top-right corner: small ⓘ icon. On hover shows a tooltip:
  tooltip text = props.source
  tooltip style: dark bg, white text, 12px font, absolute positioned
  use CSS :hover — no JS needed

Number display: 28px bold, colour = props.color
Label: 13px #666666 below number
Subtext: 12px coloured text (green if contains ↑, red if ↓, grey otherwise)

Count-up animation: create src/utils/useCountUp.js
  export default function useCountUp(target, duration=1200, enabled=true)
  Uses requestAnimationFrame. Eases out (ease = t => 1 - Math.pow(1-t, 3)).
  Returns current display value (integer).
  If target is not a number (e.g. '$56,290'), skip animation and return as-is.
```

When `animateOnMount=true`, value counts up from 0 on first render.

— COMPONENT 2: *FunnelChart.jsx* —

Props: `stages` (array of {`stage`, `count`, `color`}), `title` (string), `orientation` ('vertical'|'horizontal', default 'vertical')

Render a custom SVG funnel (NOT Recharts – custom SVG for full control):

- Each stage is a trapezoid shape
- Width scales proportionally: widest bar = widest stage (largest count)
- Height per stage: 44px
- Gap between stages: 4px
- Left side: stage name (13px, #444)
- Right side: count + conversion % from previous stage
- Conversion %: $((\text{current}/\text{previous}) * 100).toFixed(0) + \%$ – show in small grey text
- On click: call `props.onStageClick(stage)` if provided
- Hover: lighten the bar fill by 15%
- Animate bar width from 0 to final on mount (CSS transition 400ms)

— COMPONENT 3: *LeadTable.jsx* —

Props: `data` (array), `columns` (array of {`key`, `label`, `width`}), `filters` (object), `onFilterChange` (fn), `searchable` (bool), `expandable` (bool)

Features:

- Search bar at top (filters by company name, case-insensitive)
- Column header click = sort ascending/descending (toggle)
- Sort indicator arrow in header
- Alternating row background (#FFFFFF / #F0F6FC)
- Status badges: colour-coded pills for stage and interest fields
stage colours: Lead=#E6F1FB/blue, Contacted=#EAF3DE/green,
Qualified=#FEF3C7/amber, Interested=#FAEEDA/orange,
LOI=#EAF3DE/darkgreen
interest: High=#EAF3DE/green, Medium=#FEF3C7/amber, Low=#FCEBEB/red
- When `expandable=true`: clicking a row toggles an expanded detail panel showing all remaining fields (phone, email, address, source URL as clickable link)
- Pagination: show 10 rows per page, prev/next buttons
- Row count summary: 'Showing X-Y of Z leads'

Export CSV button (top-right of table):

```
import Papa from 'papaparse'
On click: Papa.unparse(filteredData), download as blob URL
Filename: 'jol-leads-[timestamp].csv'
```

Ensure all three components compile with no errors and accept their props.

TASK-03**Module 1 — Supplier pipeline****M1**
~60 min

Module	M1 · Suppliers
Estimated time	~60 min
Prerequisites	TASK-02 complete · supplierLeads.js in src/data/

DELIVERS

- SupplierPipeline.jsx with 4 KPI cards
- Animated funnel chart
- City volume horizontal bar chart (Recharts)
- State heatmap (5-cell)
- Full sortable lead table with filters
- Smart stale-lead alert

CONTEXT NOTE

This is the largest module. Import supplierLeads and supplierStats from src/data/supplierLeads.js. All data comes from those exports — no hardcoding. The stale lead alert must fire when lastContactDaysAgo >= 14 AND stage === 'Lead'.

PROMPT — copy everything below this line into Claude Code

Create src/components/SupplierPipeline.jsx.

Import: { supplierLeads, supplierStats } from '../data/supplierLeads.js'

LAYOUT (top to bottom):

ROW 1 — SMART ALERT BANNER

Compute: staleLeads = supplierLeads.filter(l => l.lastContactDaysAgo >= 14 && l.stage === 'Lead')

If staleLeads.length > 0, show a yellow banner:

'⚠️ {staleLeads.length} supplier leads have not been contacted in 14+ days.
Priority cities: {top 3 cities from staleLeads by count}'

Banner: bg #FEF3C7, border-left 3px #B45309, 12px padding, dismissible (×).

ROW 2 — KPI CARDS (4-column grid)

Use KPICard component for each:

1. supplierStats.total label='Total supplier leads'
color='#185FA5'

source='TNPCB/KSPCB/CPCB/IndiaMART registries, Jun 2026'

2. supplierStats.citiesCount label='Cities covered'
color='#0A7864'

```

    source='Derived from verified company addresses'
3. supplierStats.totalKg.toLocaleString()+' kg' label='Est. monthly
pipeline' color='#B45309'
    source='Volume tier midpoint formula (see PRD $4.6)'
4. supplierStats.tonPlusTier label='1-Ton+ tier leads'
color='#A32D2D'
    source='Volume tier from registry capacity data'
All cards: animateOnMount=true

```

ROW 3 – FUNNEL + CITY CHART (60/40 split)

LEFT (60%): FunnelChart component

```

    stages = supplierStats.funnelStages (already computed in the data file)
    title = 'Supplier pipeline'
    On stage click: set activeStageFilter state, filter table below

```

RIGHT (40%): Horizontal BarChart (Recharts)

```

    data = supplierStats.cityVolumeChart (top 8 cities by kg)
    x-axis: kg value, y-axis: city name
    Bar fill: #185FA5, hover: #0C447C
    Custom tooltip: '{city}: {kg.toLocaleString()} kg/month (estimated)'
    On bar click: set activeCityFilter state, filter table below
    Chart title: 'Estimated monthly volume by city'

```

ROW 4 – STATE HEATMAP (5 cells inline)

States: Tamil Nadu (15), Karnataka (11), Telangana (7), Andhra Pradesh (3), Multi-state (2)

Render 5 inline pills. Each pill:

```

    background intensity = count/15 mapped to blue ramp (#E6F1FB → #0C447C)
    text = state abbreviation + count
    On click: filter table to that state

```

ROW 5 – FILTER BAR

Dropdowns: Battery Type | Volume Tier | Pipeline Stage | Interest | Source Registry

Each dropdown filters the lead table below.

Active filters shown as removable chips above the table.

Clear All button.

ROW 6 – LEAD TABLE

Use LeadTable component.

columns:

```

    company (220px), city (100px), state (100px), batteryType (joined with ', ', 150px),
    volTier (120px), monthlyKgEstimate (100px), stage (120px), interest (90px),
    lastContactDaysAgo (show as '{n} days ago', 110px)
    data = supplierLeads filtered by all active filters (stage, city, state, batteryType, tier, interest, search)

```

```
expandable=true: expanded row shows phone, email, address, disposalMethod,  
source + sourceUrl
```

All filter state (activeStageFilter, activeCityFilter, activeStateFilter, etc.) is managed in SupplierPipeline with useState. Filters are applied with Array.filter() before passing to LeadTable.

TASK-04**Module 2 — Buyer pipeline****M2**
~50 min

Module	M2 · Buyers
Estimated time	~50 min
Prerequisites	TASK-02 complete · buyerLeads.js in src/data/

DELIVERS

- BuyerPipeline.jsx with 4 KPI cards
- Side-by-side funnel comparison (buyer vs supplier)
- Product mix donut (Recharts PieChart)
- Buyer category bar chart
- Buyer lead table with LOI probability badges

CONTEXT NOTE

The side-by-side funnel is the visual centrepiece. Import both `supplierStats` and `buyerStats` so both funnels can render together. The LOI probability badge uses a colour gradient from red (low) to green (high).

PROMPT — copy everything below this line into Claude Code

```
Create src/components/BuyerPipeline.jsx.
Import: { buyerLeads, buyerStats } from '../data/buyerLeads.js'
Import: { supplierStats } from '../data/supplierLeads.js' # for side-by-side
funnel

# LAYOUT:

## ROW 1 — KPI CARDS (4-column)
1. buyerStats.total 'Total buyer leads' color='#0A7864'
   source='IndiaMART/company sites/industry reports, Jun 2026'
2. buyerStats.totalConsumptionMT.toLocaleString()+' MT/mo' 'Total
addressable demand' color='#185FA5'
   source='Published capacity data + dummy consumption estimates (see PRD
$4.3)'
3. buyerStats.highLoiCount 'High LOI probability (≥50%)' color='#B45309'
   source='LOI probability derived from pipeline stage (see PRD $4.6)'
4. Object.keys(buyerStats.byProduct).length 'Metal salt products'
color='#534AB7'
   source='NiSO4, CoSO4, Li2CO3, MnSO4, LiOH, PREC, FeSO4'

## ROW 2 — SIDE-BY-SIDE FUNNELS
```

Two FunnelChart components rendered side by side (50/50).

LEFT funnel:

```
title = 'Supplier pipeline (inflow)'
stages = supplierStats.funnelStages
colours = blue ramp (#B5D4F4 → #042C53)
```

RIGHT funnel:

```
title = 'Buyer pipeline (outflow)'
stages = buyerStats.funnelStages
colours = teal ramp (#9FE1CB → #04342C)
```

Between the two funnels, render a vertical divider with the text:

'JOL ENERGY' (navy, 12px, rotated 90°) in the centre

Below both funnels, a caption:

'Supply-demand balance: {supplierStats.total} inflow leads vs {buyerStats.total} off-take leads'

ROW 3 – PRODUCT MIX + CATEGORY BREAKDOWN (50/50)

LEFT: PieChart (Recharts) – product mix donut

```
data = buyerStats.productDonut [{product, count}]
innerRadius=60, outerRadius=90
Custom legend below (not Recharts legend – custom list with colour dots)
Colours: NiSO4=#185FA5, CoSO4=#0A7864, Li2CO3=#B45309, MnSO4=#534AB7,
         PREC=#A32D2D, LiOH=#0F6E56, FeSO4=#888780
Title: 'Product requirement mix'
```

RIGHT: BarChart (Recharts, vertical) – category breakdown

```
data =
Object.entries(buyerStats.byCategory).map(([cat,count])=>({cat,count}))
Category labels: SALT_MFR='Salt Mfrs', CELL_MAKER='Cell Makers',
                CAM_MAKER='CAM/Cathode', METAL_RECOV='Metal Recovery'
Bar fill by category: SALT_MFR=#B5D4F4, CELL_MAKER=#EAF3DE,
                    CAM_MAKER=#FEF3C7, METAL_RECOV=#EEEDFE
Title: 'Buyer category breakdown'
```

ROW 4 – BUYER LEAD TABLE

Use LeadTable component.

columns:

company (220px), city (110px), products (joined, 180px), category (120px),
consumptionMT (80px, shown as 'X MT/mo'), stage (140px), loiProbability
(shown as % badge)

loiProbability badge: pill with colour gradient:

0-30% = bg #FCEBEB text #791F1F

```
31-50% = bg #FEF3C7 text #B45309  
51-80% = bg #EAF3DE text #27500A  
81-100% = bg #E1F5EE text #085041
```

```
expandable=true: expanded row shows currentSupplier, procurementProcess,  
sampleRequested, lastContactDate, source
```

```
Filter dropdowns: Category | Product | Stage | LOI range
```

TASK-05**Module 3 — Collection network geo-map****M3**
~60 min

Module	M3 · Collection
Estimated time	~60 min
Prerequisites	TASK-00 (leaflet + react-leaflet installed) · collectionPoints.js in src/data/

DELIVERS

- CollectionNetwork.jsx with full Leaflet map
- Hub circles (80 km radius)
- Colour-coded pin types
- Click-to-info popup cards
- Hub economics sidebar table
- City volume bar chart
- Category donut chart

CONTEXT NOTE

Leaflet requires dynamic import (no SSR). Use MapContainer from react-leaflet. Default Leaflet marker icons break with Vite — must fix with a custom icon setup. Hub circles use L.circle() or react-leaflet Circle component.

PROMPT — copy everything below this line into Claude Code

```
Create src/components/CollectionNetwork.jsx.
Import: { collectionPoints, collectionStats, collectionHubs, hubEconomics }
        from '../data/collectionPoints.js'
```

CRITICAL LEAFLET ICON FIX

At the top of the file (or in a separate leafletIconFix.js imported here), add:

```
import L from 'leaflet'
import iconUrl from 'leaflet/dist/images/marker-icon.png'
import iconShadow from 'leaflet/dist/images/marker-shadow.png'
const DefaultIcon = L.icon({ iconUrl, shadowUrl: iconShadow,
  iconSize:[25,41], iconAnchor:[12,41], popupAnchor:[1,-34] })
L.Marker.prototype.options.icon = DefaultIcon
```

PIN TYPE ICONS

Create 4 custom marker icons with different colours using L.divIcon:

```
REGISTRY: blue circle (#185FA5)
PRO:      teal circle (#0A7864)
INDIAMART: amber circle (#B45309)
```

FIELD: grey circle (#888780) with dashed border
 Each DivIcon: 14px circle, solid background, 1px white border, border-radius 50%

LAYOUT: two columns (map 65% left, sidebar 35% right)

LEFT - LEAFLET MAP

MapContainer: center=[14.5, 78.5], zoom=6, height=520px, width=100%
 TileLayer: 'https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png'
 attribution: '© OpenStreetMap contributors'

3 hub Circle overlays (react-leaflet Circle):
 collectionHubs.forEach: center=[hub.lat, hub.lng], radius=hub.radiusKm*1000,
 pathOptions: color=hub.color, fillColor=hub.color, fillOpacity=0.05,
 weight=1.5

Hub centre Markers (larger icon):

For each hub, a Marker with a custom L.divIcon showing hub name label

Collection point Markers:

collectionPoints.forEach: Marker at [point.lat, point.lng], icon = type-specific icon

Each Marker has a Popup (react-leaflet Popup):

- Business name (bold, 14px)
- Category pill (coloured badge)
- Est. monthly volume: '{monthlyKgEstimate} kg/mo'
- Willingness pill: Yes=green, Maybe=amber, No=red
- Source: small grey italic text
- If verified: small '✓ Verified' green badge

Map legend (bottom-right corner, custom HTML overlay):

4 rows: ● REGISTRY | ● PRO | ● INDIAMART | ○ FIELD (dashed)
 + note: 'Circles = 80 km hub radius'

RIGHT - SIDEBAR

KPI cards (3 inline)

total={collectionStats.total} 'Collection points'
 verified={collectionStats.verified} 'Registry verified'
 totalKg={collectionStats.totalKg.toLocaleString()} kg 'Est. monthly capacity'

Hub economics table

Render hubEconomics as a styled table:

columns: Hub | Collections/trip | Trips/mo | ₹/kg | Breakeven kg | Est. monthly kg

Highlight lowest ₹/kg cell (Chennai) with green background.

```
### Category donut chart (Recharts PieChart)
data from collectionStats.byType
Rename types: REGISTRY='Registry', PRO='PRO Points', INDIAMART='IndiaMART',
FIELD='Field Survey'
Size: 160px × 160px, innerRadius=40

### City volume bar chart (Recharts BarChart, horizontal)
data from collectionStats.cityVolume (top 7)
Bar fill: #B45309 (amber)
Title: 'Est. monthly volume by city (kg)'

### Pin type legend with counts
Bullet list: '17 Registry/PRO points (verified)' + '15 Field survey points (to
visit)'
Note: 'Field survey points marked with dashed pins – visit to verify'
```

TASK-06**Module 4 — Market intelligence + margin calculator****M4**
~60 min

Module	M4 · Market
Estimated time	~60 min
Prerequisites	TASK-00 complete · marketPrices.js in src/data/

DELIVERS

- MarketIntelligence.jsx with 30-day multi-line price chart
- Interactive margin calculator widget
- Battery scrap reference table
- Policy tracker with status pills
- Currency tile with USD/INR rate

CONTEXT NOTE

The margin calculator is the centrepiece of this module — it must be fully interactive with sliders. The price chart needs toggleable metal lines. Use *Recharts LineChart*. The *calcRecoveryValueINR* function is already exported from *marketPrices.js*.

PROMPT — copy everything below this line into Claude Code

```
Create src/components/MarketIntelligence.jsx.
Import: { priceHistory, currentPrices, scrapPriceRef, policyTracker,
          chemistryProfiles, calcRecoveryValueINR, marketStats, sparklines }
        from '../data/marketPrices.js'

# LAYOUT (top to bottom):

## ROW 1 — METAL PRICE KPI CARDS (4 across)
For each metal in currentPrices.metals:
  KPICard: value='${metal.usdPerMT.toLocaleString()}/MT'
           subtext='₹{metal.inrPerKg.toLocaleString()}/kg · {metal.trend}'
           color=metal.color
           source=metal.source + ' — ' + metal.sourceUrl
  Below the KPICard, render a sparkline (7-day mini chart):
    Use Recharts LineChart: width=100%, height=40, no axes, no grid
    data = sparklines[metal.id] mapped as [{v: price}]
    Line: stroke=metal.color, strokeWidth=1.5, dot=false
    The sparkline sits directly below the card with no margin

## ROW 2 — 30-DAY PRICE CHART (full width)
Recharts LineChart: width=100%, height=280
```

```
data = priceHistory (30 records, each has dateShort, cobalt_usd, nickel_usd,
liCarb_usd)
```

Toggle buttons above chart (3 pills):

```
[● Cobalt] [● Nickel] [● Li Carbonate]
```

Active = filled, inactive = outline. Clicking toggles that line.

Default: all three active.

```
XAxis: dataKey='dateShort', tickCount=6, 12px
```

```
YAxis: left axis for cobalt (0-70000), right axis for nickel+liCarb (0-20000)
      (dual axis because cobalt price is much higher)
```

Three Line components:

```
cobalt_usd: stroke='#185FA5', name='Cobalt (USD/MT)'
```

```
nickel_usd: stroke='#0A7864', name='Nickel (USD/MT)' yAxisId='right'
```

```
liCarb_usd: stroke='#B45309', name='Li Carb (USD/MT)' yAxisId='right'
```

Custom Tooltip showing:

```
Date: {date}
```

```
For each active metal: '{name}: ${value.toLocaleString()}'
```

```
(₹{(value/1000*95).toFixed(0).toLocaleString()}/kg)'
```

```
Source citation in grey italic at bottom
```

Chart title: '30-day battery metal spot prices · Source: Trading Economics / SMM'

ROW 3 – MARGIN CALCULATOR + SCRAP TABLE (55/45 split)

LEFT (55%): MarginCalculator section

Title: 'Recovery margin calculator'

Subtitle: 'Estimate monthly net recovery value from battery feedstock'

Three inputs:

1. Input volume slider: 100–5000 kg, step=100

Label: 'Monthly input volume: {value.toLocaleString()} kg'

2. Chemistry selector (3 radio buttons or segmented control):

NMC 622 | NMC 811 | LFP

Show note below: chemistry.note (from chemistryProfiles)

3. Processing cost slider: ₹10–₹60/kg, step=5

Label: 'Processing cost: ₹{value}/kg'

Output box (prominent card, dark blue background #0D2137 white text):

Call calcRecoveryValueINR(selectedChemistry, inputKg, processingCost)

Show:

```
'Gross recovery value: ₹{result.grossINR.toLocaleString()}'  
'Processing cost:      -₹{result.processingCostINR.toLocaleString()}'  
separator line  
'Net recovery value:   ₹{result.netINR.toLocaleString()}' (24px, bold)  
'= ₹{result.netPerKgINR.toLocaleString()}/kg input' (14px, amber)
```

Note below: 'Based on latest prices. Real margins depend on battery condition,
actual recovery yield, and market pricing at time of sale.'

RIGHT (45%): Scrap price reference table

Title: 'Battery scrap price reference'

Table columns: Category | Avg weight | INR/kg range | Tier
data = scrapPriceRef

Each row: left border coloured with scrapPriceRef[i].color

Tier pill: High=green, Medium=amber, Low=grey

ROW 4 – POLICY TRACKER (full width)

Title: 'Policy & regulatory tracker'

4 cards in a 2x2 grid, one per policyTracker entry.

Each card:

- Left border: 3px solid green (#27500A)
- Status pill: 'Active' green pill
- Title: bold 14px
- Summary: 13px grey, 2 lines max then truncate
- Impact line: 'Impact: {impact}' in 12px with colour matching severity
- Source link: '{authority}' as small clickable text (opens sourceUrl)

ROW 5 – CURRENCY TILE (right-aligned small card)

Show: 'USD/INR: ₹{currentPrices.usdInr} · As of {currentPrices.asOf}'

Subtext: 'Source: exchange-rates.org'

Refresh button: on click, shows toast 'Rate updated to ₹95.0 (demo – connect live API for production)'

TASK-07**Module 5 — AI insights engine (Claude API)****M5**
~60 min

Module	M5 · AI
Estimated time	~60 min
Prerequisites	TASK-00 complete (.env with VITE_ANTHROPIC_KEY) · All data files in src/data/

DELIVERS

- AllInsights.jsx with 4 sections
- Pipeline health narrative (auto-generated)
- City recommendation engine
- Smart alerts panel
- Ask-your-data chat interface with streaming

CONTEXT NOTE

Use `import.meta.env.VITE_ANTHROPIC_KEY` for the API key. Use streaming fetch with `ReadableStream` for the chat panel typewriter effect. Rate-limit to 1 call per module view. Include a graceful fallback for when the API key is missing or the call fails.

PROMPT — copy everything below this line into Claude Code

Create `src/components/AIInsights.jsx`.

Import all four data exports needed to build the pipeline context:

```
import { supplierLeads, supplierStats } from '../data/supplierLeads.js'
import { buyerLeads, buyerStats } from '../data/buyerLeads.js'
import { collectionStats } from '../data/collectionPoints.js'
import { marketStats } from '../data/marketPrices.js'
```

SHARED CONTEXT BUILDER

Create a function `buildPipelineContext()` that returns a JSON string:

```
{
  supplierStats: { total, byState, funnelStages, totalKg, tonPlusTier,
    staleLeads,
    highInterest, citiesCount, cityVolumeChart (top 5) },
  buyerStats: { total, byCategory, byProduct, totalConsumptionMT,
    highLoiCount },
  collectionStats: { total, verified, field, totalKg, byCity },
  market: { cobalt: marketStats.cobalt, nickel: marketStats.nickel,
    liCarb: marketStats.liCarb, latestDate: marketStats.latestDate }
}
```

Keep it under 800 tokens (summarise, don't dump raw arrays).

```
# LAYOUT (4 sections stacked):
```

```
## SECTION 1 — PIPELINE HEALTH NARRATIVE
```

```
Header: 'Pipeline health analysis' (auto-generated on module mount)
```

```
useEffect on mount: fire ONE Claude API call:
```

```
  model: 'claude-sonnet-4-20250514'
```

```
  max_tokens: 350
```

```
  system: 'You are a business analyst assistant for Jol Energy, a Li-ion battery
```

```
          recycling startup. Analyse the pipeline data and give a 2-3 sentence
```

```
          business narrative. Be specific with numbers. Be direct and actionable.
```

```
          No preamble, no headers — just the narrative paragraph.'
```

```
  user: 'Pipeline data as of Jun 2026: ' + buildPipelineContext()
```

```
While loading: show animated skeleton (3 grey bars of varying width, pulsing opacity).
```

```
On success: render the response text in a blue-bordered card.
```

```
On error / no API key: show fallback static text:
```

```
  'Your supplier pipeline has {supplierStats.total} leads across
```

```
  {supplierStats.citiesCount} cities. {supplierStats.staleLeads} leads
```

```
  in Lead stage have not been contacted in 14+ days — these are your
```

```
  top priority. {supplierStats.tonPlusTier} leads are at 1-Ton+ tier,
```

```
  representing
```

```
{(supplierStats.tonPlusTier/supplierStats.total*100).toFixed(0)}%
```

```
  of your high-value pipeline.'
```

```
(Interpolate actual values from supplierStats)
```

```
## SECTION 2 — CITY RECOMMENDATION
```

```
Header: 'Recommended next target city'
```

```
Fire ONE Claude API call on mount (same batch as section 1 if possible):
```

```
  max_tokens: 200
```

```
  system: 'You are a business development advisor for Jol Energy.
```

```
          Recommend the single best city to prioritise for supplier outreach
```

```
          next week. Give ONE city and ONE specific reason based on the data.
```

```
          Format: CITY: [name] — [one sentence reason]. No other text.'
```

```
  user: buildPipelineContext()
```

```
Render the response in an amber callout card.
```

```
Fallback (no API): compute from data — city with most 1-Ton+ tier leads
```

```
  AND lowest contact rate. Show: 'Recommended: {city} — {count} high-value leads
```

```
  with low contact rate.'
```

SECTION 3 — SMART ALERTS PANEL

Header: 'Smart alerts ({alertCount} active)'

Rule-based alerts — computed from data, NO API call needed:

```

alert1: staleLeads > 0 →
  '{staleLeads.length} supplier leads not contacted in 14+ days'
  severity: warn
alert2: cobalt 30d change > 10% →
  'Cobalt up {change}% in 30 days — NMC recovery margin impact:
+ ₹{impact}/kg'
  compute impact: (change/100) * 0.12 * cobaltPrice/1000 * 95
  severity: info
alert3: collectionStats.field > collectionStats.verified →
  '{collectionStats.field} collection points are unverified field
placeholders
  — schedule ground surveys in Chennai and Bengaluru'
  severity: warn
alert4: buyerStats.highLoiCount >= 5 →
  '{buyerStats.highLoiCount} buyers have ≥50% LOI probability
  — prioritise formal proposal to Nicomet, Epsilon, and Opera Chemisol'
  severity: tip

```

Alert card design:

```

warn = left border #B45309 amber + bg #FEF3C7
info  = left border #185FA5 blue + bg #D6E8F7
tip   = left border #0A7864 teal + bg #E0F4EF
Each card: icon + message + 'Dismiss' button

```

SECTION 4 — ASK YOUR DATA (CHAT)

Header: 'Ask your pipeline data'

Subtitle: 'Ask anything about your suppliers, buyers, or market prices'

Suggested questions as clickable chips (clicking pre-fills the input):

```

'Which city has the highest estimated monthly volume?'
'How many leads are at 1-Ton+ tier?'
'What does the cobalt price trend mean for NMC margins?'
'Which buyer has the highest consumption?'

```

Chat input: text input + Send button

On send:

1. Append user message to chat history
2. Show typing indicator (3 animated dots)
3. Call Claude API with STREAMING:

```

model: 'claude-sonnet-4-20250514'
max_tokens: 400
stream: true
system: 'You are a data assistant for Jol Energy. Answer ONLY based on
the

```

```

        pipeline data provided. Be concise (max 3 sentences). Include
        specific numbers from the data. If you cannot answer from the
data,
        say so briefly.'

```

```

user: 'Data: ' + buildPipelineContext() + '\n\nQuestion: ' + userMessage

```

```

4. Stream the response character-by-character into the chat bubble
   Use fetch with ReadableStream and TextDecoder for streaming.
   Append each chunk to the assistant message in state.

```

```

5. On completion: scroll chat to bottom

```

```

6. On error: show 'API unavailable – check VITE_ANTHROPIC_KEY in .env'

```

Chat bubble styles:

User: right-aligned, bg #185FA5, white text, 8px radius

Assistant: left-aligned, bg #F0F6FC, dark text, 8px radius

Chat container: max-height 320px, overflow-y auto

STREAMING IMPLEMENTATION

The Anthropic API returns server-sent events. Parse like this:

```

const response = await fetch('https://api.anthropic.com/v1/messages', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'x-api-key': import.meta.env.VITE_ANTHROPIC_KEY,
    'anthropic-version': '2023-06-01',
  },
  body: JSON.stringify({ model, max_tokens, stream: true, system, messages
}))
const reader = response.body.getReader()
const decoder = new TextDecoder()
while (true) {
  const { done, value } = await reader.read()
  if (done) break
  const chunk = decoder.decode(value)
  const lines = chunk.split('\n').filter(l => l.startsWith('data: '))
  for (const line of lines) {
    const data = JSON.parse(line.slice(6))
    if (data.type === 'content_block_delta') {
      setAssistantMessage(prev => prev + data.delta.text)
    }
  }
}
}

```

TASK-08**Smart alerts panel + global polish****M0**
~40 min

Module	M0 · Foundation
Estimated time	~40 min
Prerequisites	All module tasks complete (TASK-01 to TASK-07)

DELIVERS

- AlertsPanel.jsx (slide-in drawer from right)
- Animated progress bar on page load
- Count-up verified on all KPI cards
- Export CSV verified for supplier and buyer tables
- All tabs switch without console errors

CONTEXT NOTE

This task is purely polish and wiring. No new data logic. Focus on transitions, the slide-in drawer, and ensuring the alert bell in the top bar opens the panel and shows the count badge.

PROMPT — copy everything below this line into Claude Code

Create `src/components/AlertsPanel.jsx` and wire it into `App.jsx`.

```
# AlertsPanel.jsx
```

```
Props: isOpen (bool), onClose (fn), alerts (array of {id,message,severity,dismissed})
```

Render a side drawer that slides in from the right:

Width: 320px

Position: Use CSS transform `translateX(100%)` when closed, `translateX(0)` when open

Transition: transform 250ms ease-out

Do NOT use `position:fixed` — use a normal-flow wrapper with `min-height`

Backdrop: semi-transparent overlay div behind the panel (click to close)

Panel header: `'Smart alerts ({undismissed count})' + × close button`

Alert list (same cards as M5 section 3 — reuse the alert data):

Import the same alert computation logic (or accept alerts as props from `App.jsx`)

Each alert: dismiss button updates dismissed state

Empty state: `'No active alerts — pipeline looks healthy ✓'` (green text)

```
# WIRE INTO App.jsx
1. Add alertsOpen state (default false)
2. Alert bell button in top bar: onClick = () => setAlertsOpen(true)
3. Badge on bell: count of undismisssed alerts (red circle, absolute
positioned)
4. Render <AlertsPanel isOpen={alertsOpen} onClose={()=>setAlertsOpen(false)}
/>
    as the last child of the App root div
```

```
# PAGE LOAD PROGRESS BAR
```

In App.jsx useEffect on mount:

Create a div absolutely positioned at top:0 left:0 height:3px bg:#185FA5

Animate width: 0% → 100% over 1200ms using CSS animation

Remove after animation completes

```
# POLISH CHECKLIST – verify each item:
```

- [] All 5 tabs switch without console errors
- [] KPI card count-up animations fire on tab switch (not just page load)
Fix: reset the useCountUp hook when the parent component mounts
by using a key prop tied to the active tab
- [] Export CSV button works in SupplierPipeline and BuyerPipeline
Test: click Export, verify .csv downloads with correct column headers
- [] Map pins render in CollectionNetwork without grey box (Leaflet icon fix)
- [] Margin calculator updates live as sliders move (no lag)
- [] AI chat: if VITE_ANTHROPIC_KEY is empty or 'your_key_here',
show a banner: 'Add your Anthropic API key to .env to enable AI insights'
with a link to <https://console.anthropic.com>
- [] All tab content is scrollable if it exceeds viewport height
- [] No layout overflow on 1280px viewport width

Run npm run build. Fix any TypeScript or build errors.

Report the final bundle size.

TASK-09**Utility: health score, formatters, export CSV****M0**
~20 min

Module	M0 · Foundation
Estimated time	~20 min
Prerequisites	TASK-00 complete

DELIVERS

- src/utls/healthScore.js
- src/utls/formatters.js
- src/utls/exportCsv.js

CONTEXT NOTE

These utilities are imported by multiple components. Build them first if needed, or run this task to verify they are correct after the module tasks are done.

PROMPT — copy everything below this line into Claude Code

Create three utility files in src/utls/.

```
# FILE 1: src/utls/healthScore.js
export function computeHealthScore(supplierStats) {
  const { total, funnelStages, totalKg } = supplierStats
  if (!total) return 0
  const contacted = funnelStages.find(s=>s.stage==='Contacted')?.count || 0
  const qualified = funnelStages.find(s=>s.stage==='Qualified')?.count || 0
  const interested = funnelStages.find(s=>s.stage==='Interested')?.count || 0
  const loi = funnelStages.find(s=>s.stage==='LOI Received')?.count || 0

  const advancedCount = contacted + qualified + interested + loi
  const contactRate = advancedCount / total // 0-1
  const qualRate = (qualified+interested+loi) / Math.max(advancedCount,1) // 0-1
  const coverageScore = Math.min(totalKg / 10000, 1) // 0-1
  const velocityScore = 0.6 // baseline

  const score = contactRate*30 + qualRate*25 + velocityScore*20 + coverageScore*25
  return Math.round(score)
}
```

```
# FILE 2: src/utils/formatters.js
export const fmtINR = (n) =>
  '₹' + Math.round(n).toLocaleString('en-IN')

export const fmtUSD = (n) =>
  '$' + Math.round(n).toLocaleString('en-US')

export const fmtKg = (n) =>
  n >= 1000 ? (n/1000).toFixed(1) + ' T' : Math.round(n) + ' kg'

export const fmtDaysAgo = (n) =>
  n === 0 ? 'Today' : n === 1 ? 'Yesterday' : n + ' days ago'

export const stageColor = (stage) => ({
  'Lead': { bg: '#E6F1FB', text: '#0C447C' },
  'Contacted': { bg: '#EAF3DE', text: '#27500A' },
  'Qualified': { bg: '#FEF3C7', text: '#B45309' },
  'Interested': { bg: '#FAEEDA', text: '#633806' },
  'LOI Received': { bg: '#E1F5EE', text: '#085041' },
  'Sample Requested': { bg: '#EEEDFE', text: '#3C3489' },
  'Discussion': { bg: '#FAECE7', text: '#712B13' },
}[stage] || { bg: '#F1EFE8', text: '#444441' })

export const interestColor = (lvl) => ({
  'High': { bg: '#EAF3DE', text: '#27500A' },
  'Medium': { bg: '#FEF3C7', text: '#B45309' },
  'Low': { bg: '#FCEBEB', text: '#791F1F' },
}[lvl] || { bg: '#F1EFE8', text: '#444441' })

# FILE 3: src/utils/exportCsv.js
import Papa from 'papaparse'

export function downloadCsv(data, filename) {
  const csv = Papa.unparse(data)
  const blob = new Blob([csv], { type: 'text/csv;charset=utf-8;' })
  const url = URL.createObjectURL(blob)
  const link = document.createElement('a')
  link.href = url
  link.download = filename + '_' + new Date().toISOString().slice(0,10) +
  '.csv'
  link.click()
  URL.revokeObjectURL(url)
}
```

TASK-10**Vercel deployment & final verification****M0**
~20 min

Module	M0 · Deploy
Estimated time	~20 min
Prerequisites	All tasks complete · TASK-08 build passes · GitHub account available

DELIVERERS

- Production build passes
- Vercel deployment live
- Public URL verified
- README.md with setup instructions

CONTEXT NOTE

Final task. The public Vercel URL is the single most important submission artefact. Verify it works in incognito mode before submitting.

PROMPT — copy everything below this line into Claude Code

Finalise and deploy the Jol Energy dashboard to Vercel.

STEP 1 — PRODUCTION BUILD

Run: npm run build

Fix any build errors (common: missing imports, undefined variables, Leaflet SSR issues).

Verify dist/ folder is created with no errors.

Run: npm run preview — verify the production build works locally.

STEP 2 — GITHUB REPO

Initialise git if not already done:

git init

git add .

git commit -m 'Jol Energy dashboard — complete implementation'

Create a new public repository on GitHub named 'jol-energy-dashboard'.

Push: git remote add origin https://github.com/[username]/jol-energy-dashboard.git

git push -u origin main

STEP 3 — VERCEL DEPLOY

Option A (Vercel CLI):

npm install -g vercel

```
vercel --prod
```

Follow prompts: link to your Vercel account, auto-detect Vite framework

Option B (Vercel UI):

Go to vercel.com → Add New Project → Import from GitHub

Select `jol-energy-dashboard` repo

Framework preset: Vite (auto-detected)

Build command: `npm run build`

Output directory: `dist`

Click Deploy

STEP 4 – ENV VARIABLE ON VERCEL

In Vercel project settings → Environment Variables:

Add: `VITE_ANTHROPIC_KEY` = [your actual key]

Scope: Production + Preview

Redeploy after adding the key.

STEP 5 – FINAL VERIFICATION CHECKLIST

Open the Vercel URL in an incognito browser window and verify:

- [] Page loads without blank screen
- [] All 5 tabs render their content
- [] Map pins are visible (not grey boxes) in the Collection tab
- [] Margin calculator sliders update the output live
- [] AI Insights tab shows the health narrative (or API key banner if key not set)
- [] Health score ring shows a number in the top bar
- [] Export CSV works (downloads a file)
- [] No 404 errors on page refresh (`vercel.json` rewrite handles this)
- [] Console shows no red errors

STEP 6 – README.md

Create `README.md` in the project root with:

Jol Energy Business Intelligence Dashboard

Live demo: [Vercel URL]

Setup

1. `npm install`
2. Add `VITE_ANTHROPIC_KEY` to `.env`
3. `npm run dev`

Data sources

- Supplier leads: TNPCB/KSPCB/CPCB authorized recycler registries
- Buyer leads: IndiaMART verified suppliers + company reports
- Metal prices: Trading Economics + SMM metal.com (Jun 2026)

Dummy data

- Pipeline stages, interest levels, last contact dates generated by seeded rules

- Full methodology: see `src/data/*.js` file comments

Dashboard modules

List all 5 modules with one-line descriptions

```
Commit the README: git add README.md && git commit -m 'Add README' && git push  
Vercel will auto-redeploy.
```

Report the final live URL.